LAPORAN PRAKTIKUM

POSTTEST 8

ALGORITMA PEMROGRAMAN DASAR



Disusun oleh:

Nama (2609106044)

Kelas (B1 ‘26)

PROGRAM STUDI INFORMATIKA

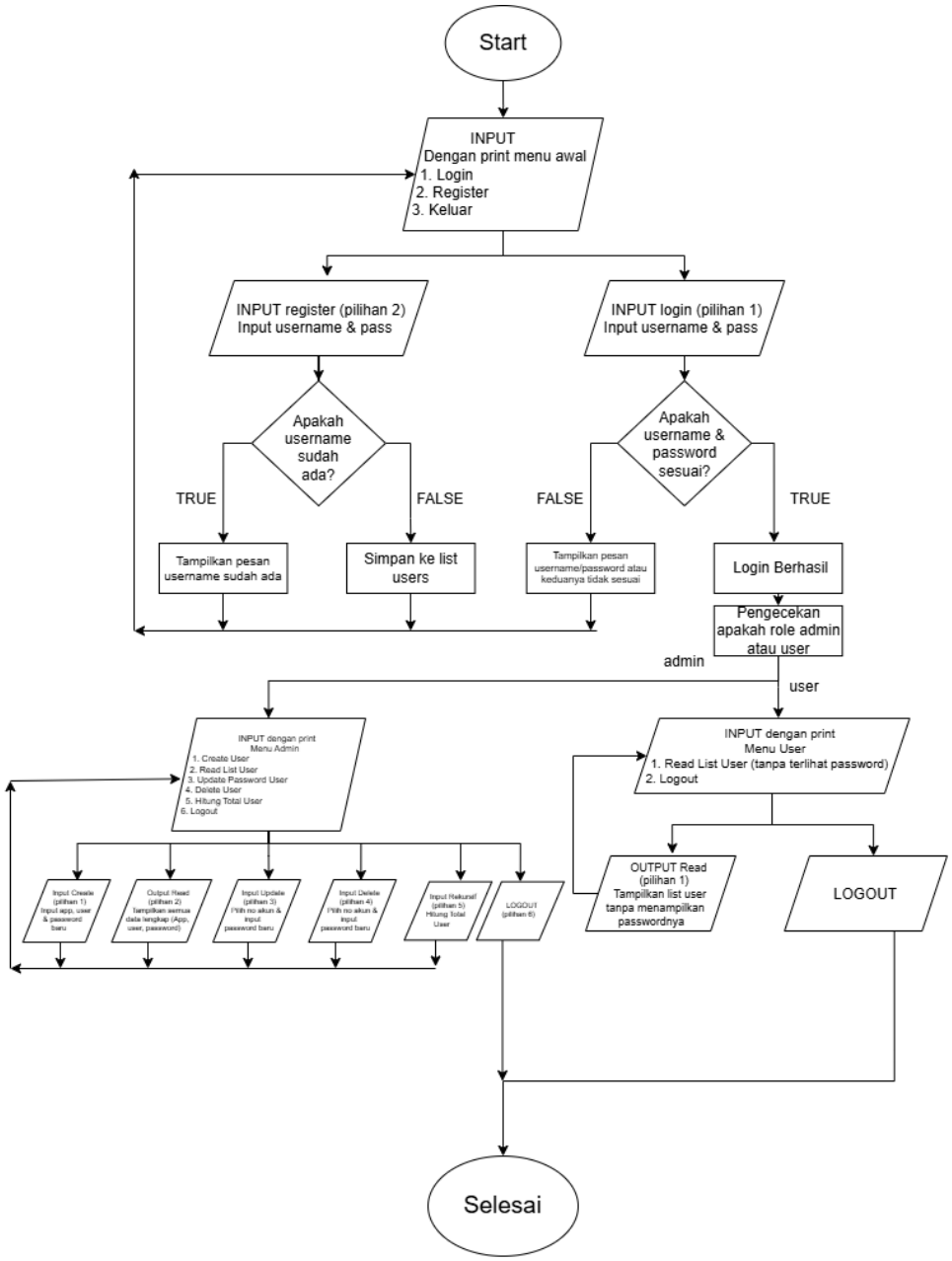
UNIVERSITAS MULAWARMAN

SAMARINDA

2026

1. Flowchart

Flowchart dimulai dari menu utama dimana pengguna diberi pilihan untuk login, membuat akun (register) atau keluar dari program. Jika pengguna memilih untuk masuk ke sistem maka pengguna akan diminta untuk memberikan kredensial yang akan diverifikasi terhadap daftar sudah terstruktur. Lalu pengguna akan dialihkan ke dashboard utama tergantung pada role. Artinya Admin akan memiliki kemampuan CRUD penuh sedangkan Pengguna Biasa (USER) hanya bisa melihat daftar aplikasi dan nama pengguna tetapi tidak bisa melihat kata sandi. Semua proses dalam dashboard dimulai dengan membersihkan layar terminal lalu memvalidasi angka yang dimasukkan tanpa kesalahan dan diakhiri dengan pilihan keluar.



2. Deskripsi Singkat Program

Program Account Manager berbasis Python berikut ini bisa membantu dalam pengelolaan akun dalam bentuk digital dengan menggunakan daftar terstruktur melalui mekanisme peran ganda yaitu Admin yang memiliki hak akses lengkap untuk melakukan operasi CRUD sementara Pengguna Biasa hanya bisa melihat nama aplikasi dan nama pengguna saja tanpa kata sandi. Alur kerja dimulai dari menu pendaftaran atau login setelah itu sistem akan membawa pengguna ke dashboard sesuai dengan peran mereka secara otomatis membersihkan layar terminal serta memvalidasi input. Namun source code dilakukan perubahan yang sebelumnya menggunakan tipe data list, sekarang menggunakan tipe data dictionary. Program kini menerapkan fungsi dengan dan tanpa parameter, prosedur, penanganan error (error handling), serta fungsi rekursif untuk menghitung total data secara otomatis. Update terbaru menerapkan sistem modular coding yaitu memisahkan beberapa fungsi program menjadi beberapa file dan mengimport fungsi-fungsi tersebut ke file utama yaitu main.py untuk menjalankan programnya.

3. Source Code

A. Database.py

File ini dibuat untuk menyimpan data user dan password yang bisa digunakan di program.

```
users = {
    "admin": {"password": "admin123", "role": "admin"},
    "user1": {"password": "user123", "role": "user"}
}

data_akun = {
    "MLBB": {"username": "Lebron_James", "password": "BronBronny"}
}

def ambil_info_sistem():
    return "Dictionary Aktif"

def hitung_total_akun_rekursif(daftar_kunci, index=0):
    if index >= len(daftar_kunci):
        return 0
    return 1 + hitung_total_akun_rekursif(daftar_kunci, index + 1)
```

B. utils.py

File ini untuk menangani pembersihan layar di terminal dan validasi input angka pilihan menu.

```
import os

def bersihkan_layar():
    os.system('cls' if os.name == 'nt' else 'clear')

def cetak_garis():
    print("==================================================")

def validasi_pilihan_menu(max_pilihan):
    try:
        input_str = input(f"Pilih menu: (1-{max_pilihan}): ").strip()
        pilihan_angka = int(input_str)
        if 1 <= pilihan_angka <= max_pilihan:
            return pilihan_angka
        else:
            print("\nPilihan menu tidak tersedia!")
            input("Tekan Enter untuk mencoba lagi.")
            return None
    except ValueError:
        print("\nError: Input harus berupa angka!")
        input("Tekan Enter untuk mencoba lagi.")
        return None
```

C. auth.py

File ini dibuat untuk mengelola autentikasi login pengguna dan pendaftaran akun baru.

```
from database import users
from utils import bersihkan_layar

def menu_login():
    bersihkan_layar()
    print("============================")
    print("         LOGIN           ")
    print("============================")
    input_user = input("Username : ").strip()
    input_pass = input("Password : ").strip()

    if input_user in users and users[input_user]["password"] ==
input_pass:
        role = users[input_user]["role"]
        print(f"\nLogin berhasil! Selamat datang {input_user}! Role kamu
'{role.upper()}' ya!")
        input("Tekan Enter untuk melanjutkan.")
        return input_user, role
    else:
        print("\nUsername atau password salah!")
        input("Tekan Enter untuk mencoba lagi.")
        return None, None

def menu_register():
    bersihkan_layar()
    print("============================")
    print("     REGISTER PENGGUNA     ")
    print("============================")
    reg_user = input("Masukkan Username Baru: ").strip()
    reg_pass = input("Masukkan Password Baru: ").strip()

    if reg_user != "" and reg_pass != "":
        if reg_user in users:
            print("\nUsername sudah terdaftar! Mohon gunakan username
lain ^^.")
        else:
            users[reg_user] = {"password": reg_pass, "role": "user"}
            print(f"\nRegistrasi berhasil! Akun '{reg_user}' telah
dibuat.")
    else:
        print("\nUsername dan password tidak boleh kosong!")
    input("Tekan Enter untuk kembali ke menu utama.")
```

D. features.py

File ini dibuat untuk mengelola operasi CRUD (Create, Read, Update, Delete) dengan penggunaan library PrettyTable untuk menampilkan data dalam bentuk tabel yang rapi.

```
from prettytable import PrettyTable
from database import data_akun, hitung_total_akun_rekursif
from utils import bersihkan_layar

def tambah_akun():
    bersihkan_layar()
    print("============================")
    print("    TAMBAH AKUN BARU       ")
    print("============================")
    app = input("Nama Aplikasi / Web : ").strip()
    uname = input("Username / Email    : ").strip()
    pwd = input("Password            : ").strip()

    if app != "" and uname != "" and pwd != "":
        if app in data_akun:
            print(f"\nAkun untuk '{app}' sudah ada! Gunakan menu Update
jika ingin mengubah.")
        else:
            data_akun[app] = {"username": uname, "password": pwd}
            print(f"\nKredensial untuk '{app}' berhasil ditambahkan!")
    else:
        print("\nMohon mengisi semua data!")
    input("Tekan Enter untuk kembali.")

def lihat_akun(role):
    bersihkan_layar()
    print("============================")
    print("        LIST AKUN           ")
    print("============================")
    if len(data_akun) == 0:
        print("Belum ada data akun yang tersimpan di database.")
    else:
        table = PrettyTable()
        if role == "admin":
            table.field_names = ["No.", "Aplikasi / Web", "Username /
Email", "Password"]
            index = 1
            for app, detail in data_akun.items():
                table.add_row([index, app, detail['username'],
detail['password']])
```

```
                index += 1
        else:
            table.field_names = ["No.", "Aplikasi / Web", "Username /
Email"]
            index = 1
            for app, detail in data_akun.items():
                table.add_row([index, app, detail['username']])
                index += 1
        print(table)
    input("\nTekan Enter untuk kembali.")

def update_password():
    bersihkan_layar()
    print("============================")
    print("   UPDATE PASSWORD AKUN    ")
    print("============================")
    if len(data_akun) == 0:
        print("Belum ada data akun yang tersimpan di database.")
    else:
        table = PrettyTable()
        table.field_names = ["No.", "Aplikasi / Web", "Username /
Email"]
        list_keys = list(data_akun.keys())
        idx = 1
        for key in list_keys:
            table.add_row([idx, key, data_akun[key]['username']])
            idx += 1
        print(table)

        input_no = input("Masukkan Nomor Akun yang ingin diubah
password-nya: ").strip()

        if input_no.isdigit():
            no_index = int(input_no) - 1
            if 0 <= no_index < len(list_keys):
                key_pilihan = list_keys[no_index]
                pass_baru = input(f"Masukkan Password Baru untuk
'{key_pilihan}': ").strip()
                if pass_baru != "":
                    data_akun[key_pilihan]["password"] = pass_baru
                    print("\nPassword berhasil diperbarui!")
                else:
                    print("\nPassword baru tidak boleh kosong!")
            else:
                print("\nNomor akun tidak ditemukan!")
        else:
```

```
            print("\nInput harus berupa angka nomor akun!")
    input("\nTekan Enter untuk kembali.")

def hapus_akun():
    bersihkan_layar()
    print("============================")
    print("        DELETE AKUN         ")
    print("============================")
    if len(data_akun) == 0:
        print("Belum ada data akun yang tersimpan di database.")
    else:
        table = PrettyTable()
        table.field_names = ["No.", "Aplikasi / Web", "Username /
Email"]
        list_keys = list(data_akun.keys())
        idx = 1
        for key in list_keys:
            table.add_row([idx, key, data_akun[key]['username']])
            idx += 1
        print(table)

        input_no = input("Masukkan Nomor Akun yang ingin dihapus:
").strip()

        if input_no.isdigit():
            no_index = int(input_no) - 1
            if 0 <= no_index < len(list_keys):
                key_pilihan = list_keys[no_index]
                akun_terhapus = data_akun.pop(key_pilihan)
                print(f"\nAkun '{key_pilihan}'
({akun_terhapus['username']}) berhasil dihapus!")
            else:
                print("\nNomor akun tidak ditemukan!")
        else:
            print("\nInput harus berupa angka nomor akun!")
    input("\nTekan Enter untuk kembali.")

def hitung_total_akun():
    bersihkan_layar()
    print("============================")
    print("   TOTAL AKUN (REKURSIF)    ")
    print("============================")
    total_hasil = hitung_total_akun_rekursif(list(data_akun.keys()))
    print(f"Total akun tersimpan di database: {total_hasil} buah")
    print("============================")
    input("\nTekan Enter untuk kembali.")
```

D. main.py

File ini sebagai file utama untuk menjalankan alur kontrol program, dengan mengimport fitur dari beberapa modul file yang berisi fitur dan fungsi untuk menjalankan program.

```
from database import ambil_info_sistem
from utils import bersihkan_layar, validasi_pilihan_menu
from auth import menu_login, menu_register
from features import tambah_akun, lihat_akun, update_password,
hapus_akun, hitung_total_akun

def tampilkan_menu_awal():
    bersihkan_layar()
    print("============================")
    print("   ACCOUNT MANAGER - LOGIN   ")
    print("============================")
    print("1. Login")
    print("2. Register (jika belum mempunyai akun)")
    print("3. Keluar program")
    print("============================")
    print("")

def main():
    while True:
        tampilkan_menu_awal()

        pilihan_awal_angka = validasi_pilihan_menu(3)
        if pilihan_awal_angka is None:
            continue

        pilihan_awal = str(pilihan_awal_angka)

        if pilihan_awal == "1":
            logged_in_user, logged_in_role = menu_login()
            if logged_in_user is not None:
                while True:
                    bersihkan_layar()
                    print("============================")
                    print(" ACCOUNT MANAGER - MAIN MENU ")
                    print("============================")
                    print(f"Pengguna Aktif : {logged_in_user} | Info:
{ambil_info_sistem()}")
                    print("============================")

                    if logged_in_role == "admin":
```

```
                        print("1. Tambah Akun Baru (Create)")
                        print("2. Lihat Semua Akun (Read)")
                        print("3. Perbarui Password Akun (Update)")
                        print("4. Hapus Akun (Delete)")
                        print("5. Hitung Total Akun (Rekursif)")
                        print("6. Logout")
                        batas_maks_menu = 6
                    else:
                        print("1. Lihat Semua Akun (Read - Akses
Terbatas)")
                        print("2. Logout")
                        batas_maks_menu = 2

                    print("============================")

                    pilihan_menu_angka =
validasi_pilihan_menu(batas_maks_menu)
                    if pilihan_menu_angka is None:
                        continue

                    pilihan_menu = str(pilihan_menu_angka)

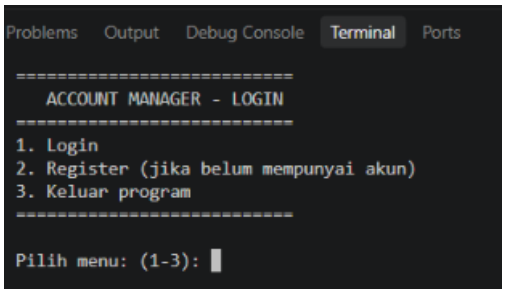
                    if logged_in_role == "admin":
                        if pilihan_menu == "1":
                            tambah_akun()
                        elif pilihan_menu == "2":
                            lihat_akun("admin")
                        elif pilihan_menu == "3":
                            update_password()
                        elif pilihan_menu == "4":
                            hapus_akun()
                        elif pilihan_menu == "5":
                            hitung_total_akun()
                        elif pilihan_menu == "6":
                            print("\nBerhasil logout dari sistem.")
                            input("Tekan Enter untuk kembali ke halaman
utama.")
                            break
                    else:
                        if pilihan_menu == "1":
                            lihat_akun("user")
                        elif pilihan_menu == "2":
                            print("\nBerhasil logout dari sistem.")
                            input("Tekan Enter untuk kembali ke halaman
utama.")
                            break
```

```
        elif pilihan_awal == "2":
            menu_register()

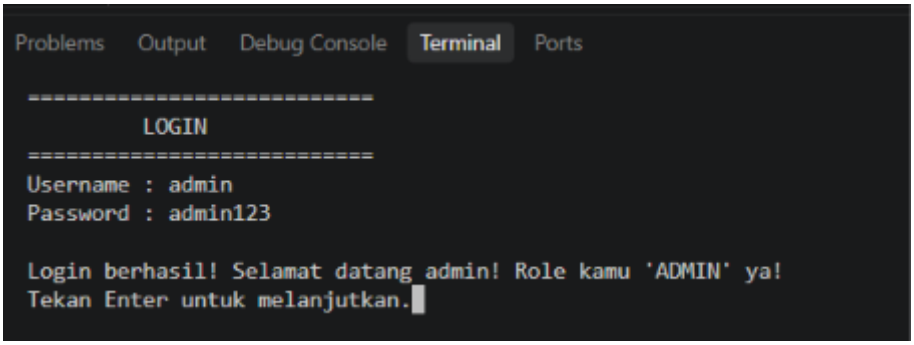
        elif pilihan_awal == "3":
            print("\nTerima Kasih sudah menggunakan program ini!")
            break

if __name__ == "__main__":
    main()
```

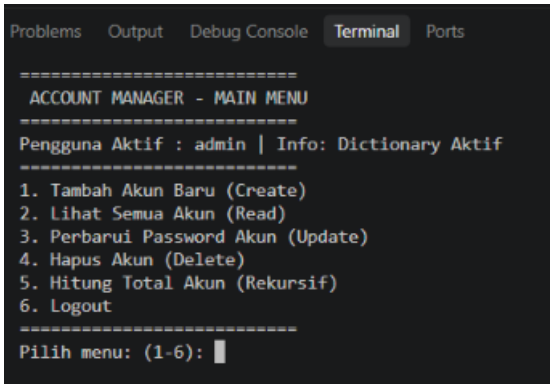
4. Hasil Output



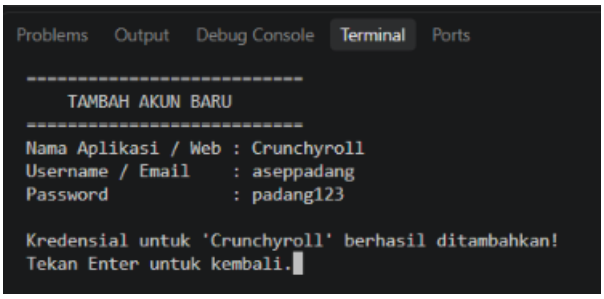
Gambar 4.1 Menu Utama Login



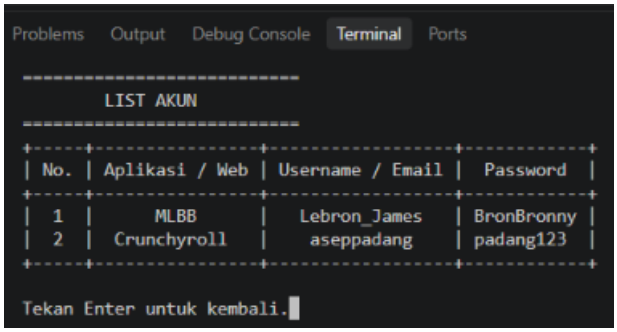
Gambar 4.2 Login



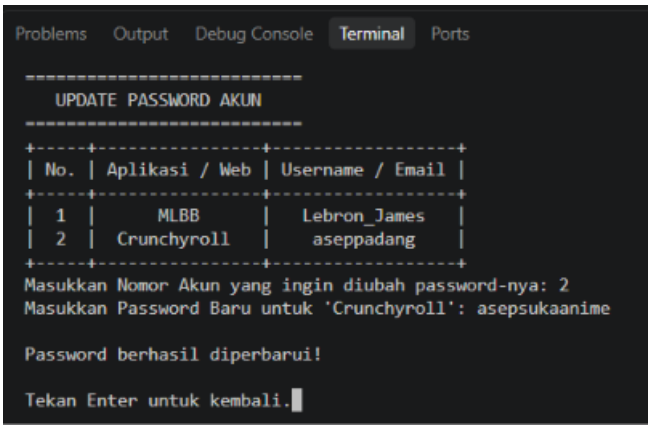
Gambar 4.3 Menu Admin



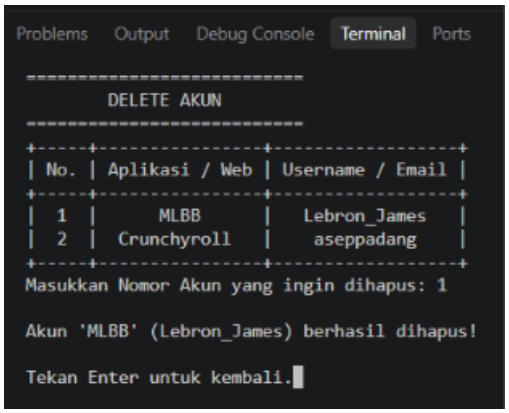
Gambar 4.4 CREATE tambah akun baru (admin)



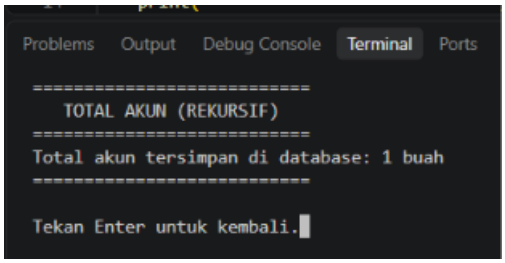
Gambar 4.5 READ list akun (admin)



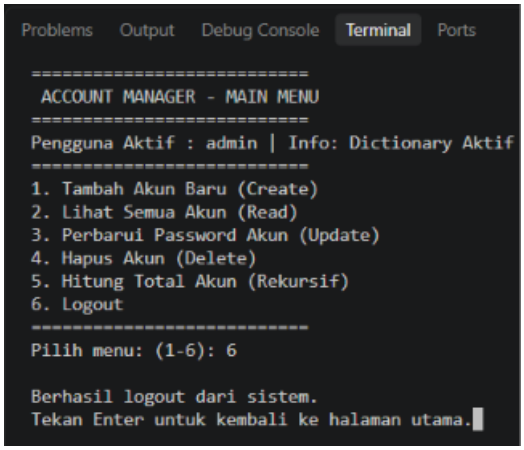
Gambar 4.6 UPDATE mengubah password akun (admin)



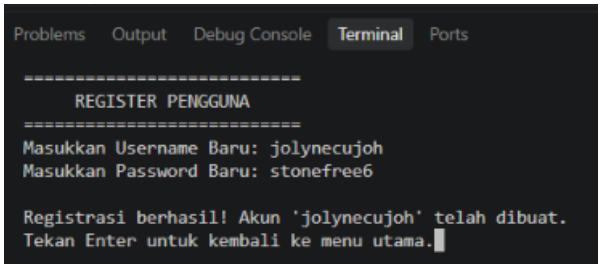
Gambar 4.7 DELETE menghapus akun (admin)



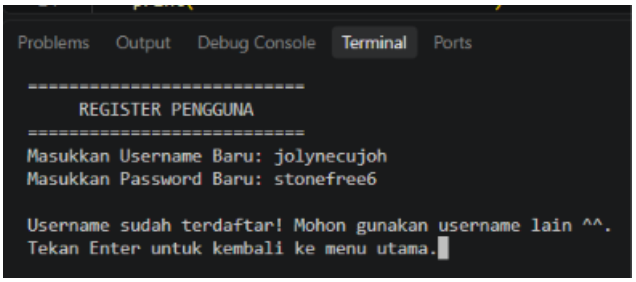
Gambar 4.7 Rekursif menghitung jumlah total akun yang ada (admin)



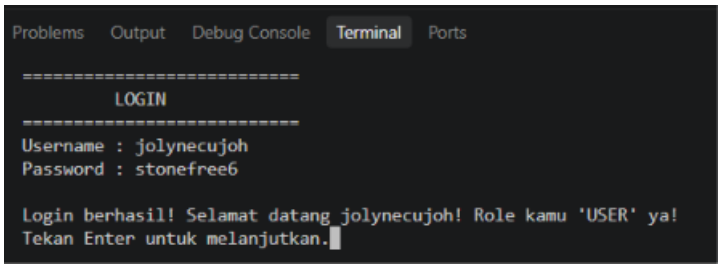
Gambar 4.8 Logout (admin)



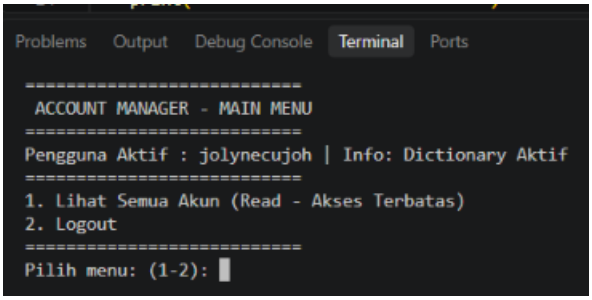
Gambar 4.9 Register Pengguna Baru



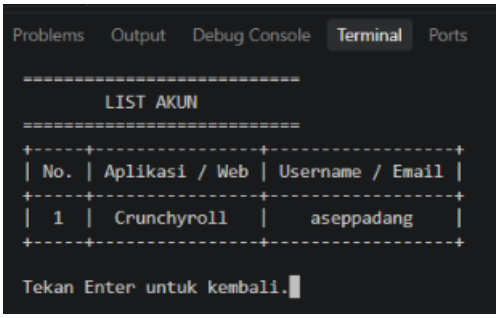
Gambar 4.10 Output jika register pengguna baru dengan user dan pw yang sudah ada



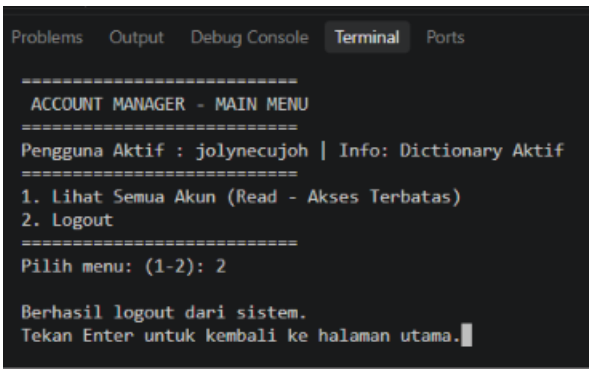
Gambar 4.11 Login sebagai USER



Gambar 4.12 Menu utama sebagai USER



Gambar 4.12 Menu READ melihat list akun sebagai USER (tidak bisa melihat password)

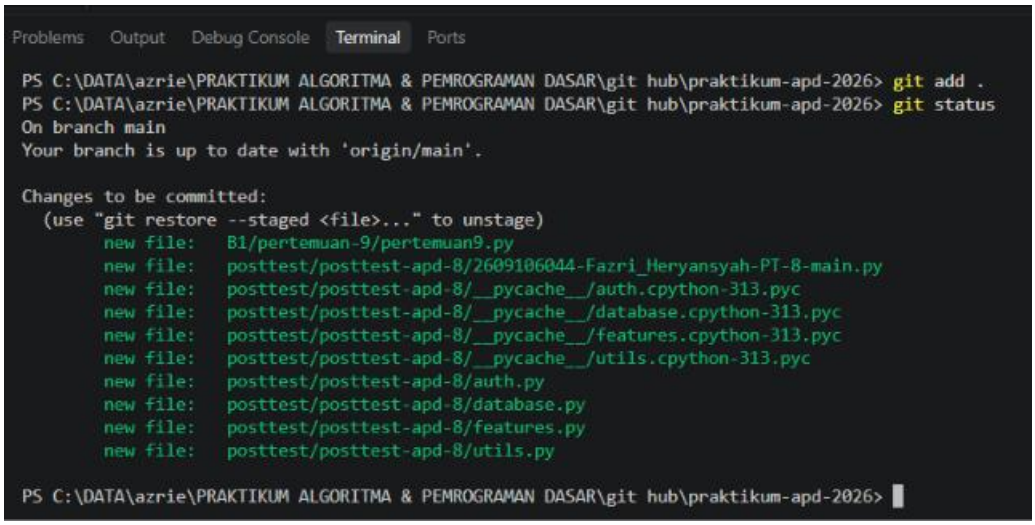


Gambar 4.12 Logout sebagai USER

5. Langkah-langkah Git

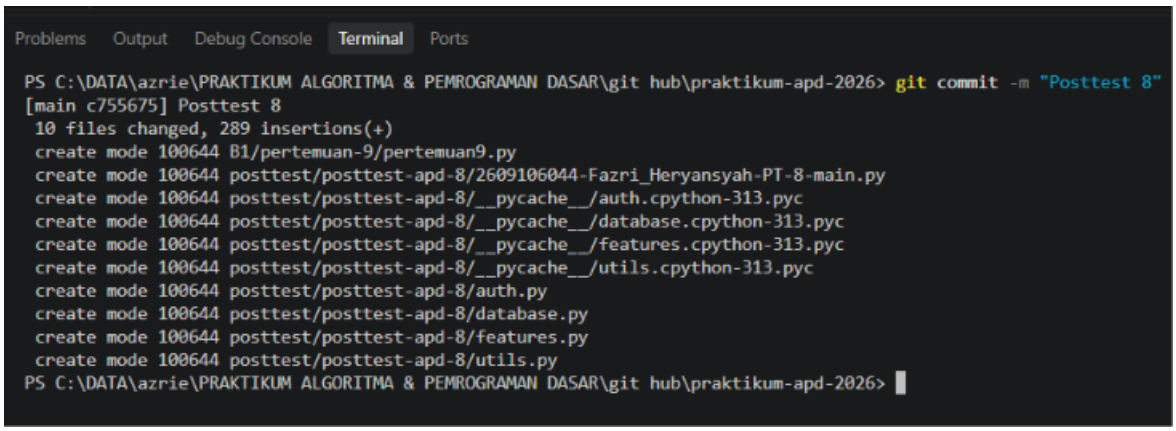
5. 1 Git Add

Perintah git add . berfungsi untuk memasukkan semua file baru, yang diubah, atau yang dihapus di direktori saat ini ke staging area (area persiapan) agar siap untuk disimpan dengan perintah.



5.2 Git Commit

Git commit berfungsi untuk merekam atau menyimpan snapshot (cuplikan) dari perubahan file yang telah disiapkan di staging area ke dalam repositori lokal.



5.3 Git Push

Git push digunakan untuk mengunggah atau mengirimkan commit (perubahan kode) dari repositori lokal di komputer ke repositori jarak jauh (remote repository).

